PROYECTO CLS · EDICIÓN 5 · REVISIÓN 5.4 · CUADERNILLO DE CONSULTA RÁPIDA

Separata de Escritorio y Referencia Rápida

Cuadernillo diseñado para permanecer abierto junto al teclado (abrochado a caballete o anillado fino) mientras trabajás con IDA, Ghidra, x64dbg, ILSpy o WinDbg y leés el **Tomo I** o el **Tomo II**.

Contenido de esta Separata de Escritorio

1. Tres perfiles o rutas de estudio (Principiante, Intermedio y Avanzado) p. 3-4 · 2 págs.
2. Árbol de decisión de triage inicial («¿Por dónde empiezo?») y 10 señales p. 5-6 · 2 págs.
3. Mapa del Hilo Conductor (check_v1 → BlocLab) y comandos del Kit C/C++ p. 7-8 · 2 págs.
4. Hoja de Referencia 1 · Nivel Principiante (Registros, saltos, VA/RVA, ABIs) p. 9-10 · 2 págs.
5. Hoja de Referencia 2 · Nivel Intermedio (Offsets PEB/LDR, cripto, x64dbg, Frida) p. 11-12 · 2 págs.
6. Hoja de Referencia 3 · Nivel Avanzado (C++ RTTI/SSO, CTL_CODE, IRQL, WinDbg/TTD) p. 13-14 · 2 págs.
7. Hoja de Referencia 4 · Evasión User-Mode, Inyección, Go/Rust y x64dbg (Rev. 5.4) p. 15 · 1 pág.
8. Hoja de Referencia 5 · Criptografía Nativa (CryptoAPI/CNG) y Drivers .sys (Rev. 5.4) p. 16 · 1 pág.

Índice Maestro Cruzado: Localización rápida en Tomo I, Tomo II y Edición Completa

Bloque / Etapa del curso	Nivel principal	Volumen impreso	Páginas en Tomo I / II	Páginas en Ed. Completa
Apertura y Mapa del curso (`00_EMPEZAR_AQUI`, `index`)	Todos	**Tomo I**	p. 3–13	p. 3–13
Etapa 0 · Laboratorio seguro (`0.1`)	Principiante	**Tomo I**	p. 14–20	p. 14–20
Etapa 1 · Fundamentos binarios y PE (`1.1–1.5`)	Principiante	**Tomo I**	p. 21–55	p. 21–55
Etapa 2A · Núcleo x86/x64, IDA y x64dbg (`2.1–2.9`)	Principiante	**Tomo I**	p. 56–121	p. 56–121
Etapa 2B · Ofuscación, TLS, MFC, PEB, Anti-debug, Ghidra (`2.10–2.16`)	Intermedio	**Tomo I**	p. 122–165	p. 122–165
Etapa 2C · Packers, C++ profundo, WinDbg/TTD, Diffing, Frida (`2.17–2.21`)	Interm. / Avanzado	**Tomo I**	p. 166–194	p. 166–194
Etapa 2 Optativas · Malware dinámico, Shellcode, Fuzzing (`2.22–2.24`)	Interm. / Avanzado	**Tomo I**	p. 195–205	p. 195–205
Etapa 3 · Reversing en .NET e IL (`3.1–3.2`)	Principiante	**Tomo I**	p. 206–219	p. 206–219
Etapa 4 · Método de 5 pasos, protocolos, cripto e informe (`4.1–4.5`)	Princ. / Intermedio	**Tomo I**	p. 220–256	p. 220–256
Etapa 5 · Malware estático, vulnerabilidades, automatización (`5.1–5.4`)	Interm. / Avanzado	**Tomo I**	p. 257–283	p. 257–283
Etapa 6 · Kernel y drivers KMDF/WDM en Windows (`6.1–6.6`)	Avanzado	**Tomo II**	p. 3–40	p. 284–321
Rama 2D · Windows internals y Hyper-V (`2D.1–2D.13` + Hyper-V)	Interm. / Avanzado	**Tomo II**	p. 41–124	p. 322–405
Hilo conductor multi-nivel y Kit de 5 plantillas C/C++	Todos	**Tomo I**	p. 284–292	p. 406–414
Hojas de referencia rápida (Cheat Sheets 1 a 5)	Todos	**Separata y Tomo I**	p. 293–298	p. 415–420
Glosario, Preguntas frecuentes y Fuentes/límites	Todos	**Tomo I**	p. 299–327	p. 421–449

Tres perfiles o rutas de estudio (Principiante, Intermedio y Avanzado)

Aunque el curso completo supera las 400 páginas al sumar el tronco obligatorio (Etapas 0–4), los bloques intermedio y avanzado de la Etapa 2 (2B y 2C), la rama de Windows internals (2D) y la de Kernel y drivers (Etapa 6), **no hace falta leerlo de corrido de principio a fin en una sola sentada**. Podés recorrerlo en tres hitos progresivos claramente delimitados según tu nivel actual y tu objetivo:

Perfil y nivel	Carga estimada	Secuencia exacta de etapas y lecciones	Herramientas centrales	Hito práctico al cerrar el nivel
Ruta 1 · Fundamentos Firmes *(Nivel Principiante)*	~65 a 75 h	Etapa 0 (0.1) → Etapa 1 (1.1–1.5) → Etapa 2A (2.1–2.9) → Etapa 3 (3.1–3.2) → Etapa 4 base (4.1, 4.2, 4.3 y 4.4)	Visor PE (PE-bear / DIE), editor hex, IDA Free, x64dbg, ILSpy / dnSpyEx, compilador C y Python	Analizar un binario propio en C y uno en .NET de punta a punta sin ver el fuente, confirmar hipótesis con breakpoints y redactar un informe reproducible.
Ruta 2 · Analista de Binarios y Malware Defensivo *(Nivel Intermedio)*	+45 a 55 h *(acumulado ~115–130 h)*	Bloque **2B** (2.10–2.16) → 2.17 (Packers) → 2.21 (Scripting y Frida) → 4.5 (Formatos y protocolos) → 5.1 (Malware estático) y 5.3 (Automatización) → Optativas 2.22, 2.23 y 2D.1–2D.5	Ghidra, Scylla, trazas condicionales de x64dbg, Frida, IDAPython (`ida_typeinf`), `pefile`, `capstone`, YARA y Sysinternals / WinDbg básico	Desempaquetar un binario UPX propio (x86/x64) hasta el OEP, recuperar strings ocultas y APIs por PEB/hash, reconstruir un formato binario propio (`CLSF`) y escribir reglas YARA e informes de persistencia.
Ruta 3 · Investigador de Vulnerabilidades, Internals y Kernel *(Nivel Avanzado)*	+80 a 110 h *(curso completo)*	Núcleo duro de **2C** (2.18 C++ profundo, 2.19 WinDbg/TTD/Heap, 2.20 Patch diffing) → 5.2 (Vulnerabilidades) → 2.24 (Fuzzing y simbólico) → 2D.6–2D.13 (COM/RPC/WMI/IPC/Excepciones/ETW) → Etapa 6 (6.1–6.6, Kernel y drivers)	WinDbg Preview + Time-Travel Debugging (TTD), GFlags / Page Heap, BinDiff / Diaphora / Version Tracking, OleViewDotNet, WDK / KMDF y Driver Verifier en VM descartable	Reconstruir clases C++ (vtable, RTTI, SSO, excepciones x64), diagnosticar corrupciones de heap con Page Heap/TTD, aislar un parche de seguridad con diffing, auditar canales IPC/COM y revisar un driver `.sys` estática y dinámicamente.

Consejo para elegir tu punto de entrada

Si empezás desde cero, completá primero la **Ruta 1** sin desviarte a 2B ni a 2C cuando llegues al final de 2.9: saltar de 2.9 directo a la Etapa 3 (.NET) y a la Etapa 4 base te da una victoria real y consolida el método en la mitad de tiempo. Una vez aprobada la puerta de salida de la Etapa 4, volvé para cursar la **Ruta 2** y luego la **Ruta 3**. En cualquier nivel, acompañate con el Hilo conductor multi-nivel y kit de plantillas C/C++ y las Hojas de referencia rápida (Cheat Sheets).

Árbol de decisión de triage inicial («¿Por dónde empiezo?»)

Cuando te enfrentás a un binario desconocido (un crackme, un programa propio compilado meses atrás o un componente de laboratorio), los primeros diez minutos del **Paso 2 (Inventario)** definen qué herramientas abrir y a qué lección del curso acudir. Usar IDA sobre un ensamblado .NET puro o buscar strings en un binario empaquetado con UPX hace perder horas. Este árbol de decisión conecta los tres niveles del curso (Principiante, Intermedio y Avanzado) en un flujo único (y se complementa con el Hilo conductor multi-nivel y kit de plantillas C/C++):



Nivel	Señal observada en los primeros 5 minutos	Qué significa (hipótesis inicial)	Acción inmediata y lección de referencia
Principiante	Imports claros (`scanf`, `printf`, `MessageBoxW`) y strings visibles en `.rdata`	Binario C/C++ sin empaquetar ni ofuscar	Buscar la string clave con `Shift+F12`, saltar por xref (`X`) y aplicar el ciclo de 5 pasos (2.7, 4.1).
Principiante	Las strings de éxito y error están en el **mismo bloque básico** sin salto condicional entre ellas	Selección sin salto (*branchless*) producida por el compilador (`-O2`)	Buscar una instrucción `CMOVcc` (`cmovne`/`cmove`) que elige qué puntero de cadena pasar a `puts`/`printf` (2.4, 2.8, 4.1).
Principiante	Importa `mscoree.dll` (`_CorExeMain`) o contiene metadatos `.NET` / marcador `.net bundle`	Aplicación administrada .NET (IL, ReadyToRun o Single-File)	Abrir en **ILSpy** o **dnSpyEx** para leer C# e IL antes de tocar un desensamblador nativo (3.1, 3.2).
Intermedio	Secciones `UPX0` (`RawSize = 0`) y `UPX1` (entropía > 7,5, permisos `W+X`), pocos imports	Ejecutable empaquetado con UPX	Llegar al OEP (*ESP trick* en x86 o breakpoint de memoria en ejecución sobre `UPX0` en x64), dumpear y reconstruir IAT con Scylla (2.17).
Intermedio	Mi breakpoint en `main` nunca se alcanza, o una variable global ya llega alterada al entrar a `main`	Código ejecutado antes del entry point principal	Revisar el directorio **TLS (`AddressOfCallBacks`)** en PE-bear/IDA y poner breakpoint en cada callback (2.11, 2.15).
Intermedio	Casi sin imports ni strings, pero lee `gs:[60h]` → `[rax+18h]` → `[rax+20h]` y rota enteros (`ror edx, 0Dh`)	Resolución dinámica de APIs por recorrido del PEB y *API hashing*	Poner breakpoint en el `call reg` del resolvedor o cruzar constantes con listas de hashes de exportaciones (2.14).
Intermedio	Constantes `0x637c777b` (AES), `"expand 32-byte k"` (ChaCha20) o `0x9E3779B9` (TEA) en `.rdata`/código	Transformación criptográfica estándar o cifrado de configuración/mensajes	Etiquetar la función criptográfica por su constante mágica e interceptar su entrada/salida en vez de analizar las rondas internas (4.2, 2.21).
Avanzado	Llamadas indirectas `call qword ptr [rax+18h]` donde `rax` proviene de `[rcx]`	Despacho virtual C++ (`vtable`) o interfaz COM (`IUnknown`)	Ubicar la `vtable` en `.rdata`, leer `vtable[-1]` (`Complete Object Locator`, RVAs en x64) para recuperar el nombre de la clase (2.18, 2D.6).
Avanzado	El proceso muere dentro de `ntdll!RtlFreeHeap` lejos de donde ocurrió el error real	Corrupción de heap previa (desborde, *use-after-free* o *double free*)	Activar **Full Page Heap** (`gflags /p /enable app.exe /full`) o grabar traza **TTD** en WinDbg y retroceder con `ba w4` + `g-` (2.19).
Avanzado	Archivo `.sys` que importa `WdfVersionBind` y registra `EvtIoDeviceControl`	Driver de modo kernel basado en KMDF	Desglosar cada constante `CTL_CODE` del `switch` (DeviceType, Access, Function, Method), revisar `SDDL` y auditar tamaños mínimos y usos de `METHOD_NEITHER` (6.2, 6.4).

Mapa del Hilo Conductor Multi-Nivel (`check_v1` → `check_v2` → `BlocLab`)

La misma regla de negocio —aceptar dos enteros `a <= 50` (`0x32`), `b >= 50` (`0x32`) tales que `(a + b) * 4 == 480` (`0x1E0`)— recorre los tres niveles del curso. El código fuente completo está en el **Tomo I** (*Hilo conductor y plantillas C/C++*) y en la carpeta `plantillas_codigo/`:

Estadio y Nivel	Qué agrega sobre el binario anterior	Qué ves en IDA / Ghidra / x64dbg / WinDbg	Lecciones que conecta
Estadio 1 · Principiante `check_v1_O0.exe` vs `check_v1_O2.exe`	Compilación en C puro sin optimizar (`/Od`) frente a optimizada (`/O2`).	En `/Od`: variables en stack y 2 saltos condicionales. En `/O2`: argumentos directo en `ECX` / `EDX`, `lea eax, [rcx+rdx]`, `cmp eax, 78h` (`480/4 = 120`) y selección sin salto (`cmove rcx, rdx`).	1.5, 2.4, 2.5, 2.7, 2.8, 4.1
Estadio 2 · Intermedio `check_v2.exe` + `parser_clsf.exe`	**TLS Callback** anti-debug antes de `main`, string cifrada con `XOR 0x5A`, empaquetado UPX y archivo binario `.clsf` (12B header + body).	Con debugger sin ocultar, el TLS callback cambia `g_objetivo = 999` antes de `main`. La string `"Correcto"` no aparece en `.rdata`; se descifra en el stack antes de `puts`.	2.10, 2.11, 2.14, 2.15, 2.17, 2.21, 4.2, 4.5
Estadio 3 · Avanzado `cpp_fh4.exe` + `heap_bug_v1.exe` + `pipe_server_min.exe`	Herencia múltiple C++ (`ValidadorBloc`), `std::string` SSO, excepciones x64, bug de heap (20B en bloque de 16B), Named Pipe y despacho IOCTL en KMDF.	Ajuste de `this` (`add rcx, 10h`), umbral SSO (`cmp [rcx+18h], 10h`), `!heap -p -a` con Page Heap / `g-` en TTD, protocolo `\\.\pipe\BlocLabPipe` y tabla `CTL_CODE`.	2.18, 2.19, 2.20, 5.2, 2D.6–2D.10, 6.2, 6.4

Referencia rápida de compilación y prueba del Kit de 5 Plantillas C/C++

Archivo en `plantillas_codigo/`	Comando de compilación (MSVC / MinGW-w64)	Objetivo de práctica y comando de verificación
1. `check_evolucion.c` *(Principiante e Intermedio)*	`cl /Zi /Od check_evolucion.c /Fe:check_v1_O0.exe` `cl /Zi /O2 check_evolucion.c /Fe:check_v1_O2.exe` `cl /Zi /O1 /DMODO_INTERMEDIO check_evolucion.c /Fe:check_v2.exe`	Comparar grafo con/sin `CMOVcc` en IDA (2.8/4.1); inspeccionar directorio TLS en PE-bear y frenar en `tls_antidebug` antes de `main` (2.11/2.15). Par válido: `40 80`.
2. `cpp_rtti_sso.cpp` *(Avanzado · C++ x64)*	`cl /EHsc /GR /Zi /O1 cpp_rtti_sso.cpp /Fe:cpp_fh4.exe` `cl /EHsc /GR /Zi /O1 /d2FH4- cpp_rtti_sso.cpp /Fe:cpp_fh3.exe`	Localizar las 2 vtables de `ValidadorBloc`, leer `vtable[-1]` (RTTI), distinguir string SSO (`< 16B`) de heap (`>= 16B`) y comparar `__CxxFrameHandler4` vs `FH3` en `.pdata/.xdata` (2.18).
3. `heap_bug.c` *(Avanzado · WinDbg y Diffing)*	`cl /Zi /O1 heap_bug.c /Fe:heap_bug_v1.exe` `cl /Zi /O1 /DPARCHE_SEGURO heap_bug.c /Fe:heap_bug_v2.exe`	Activar Page Heap: `gflags /p /enable heap_bug_v1.exe /full`, correr con 20 `A` s en WinDbg y ejecutar `!heap -p -a @rcx` (2.19). Comparar `v1` vs `v2` en Diaphora/BinDiff (2.20).
4. `parser_clsf.c` *(Intermedio · Formatos y Fuzzing)*	`cl /Zi /O1 parser_clsf.c /Fe:parser_clsf.exe` Generar muestra: `parser_clsf.exe --gen muestra_A.clsf`	Reconstruir la cabecera de 12 bytes (`magic "CLSF"` , `version` , `flags` , `hdr_len=12` , `body_len` , `seq_id`) viendo cómo acota cada campo en el desensamblado (4.5 y 2.24).
5. `pipe_server_min.c` *(Intermedio/Avanzado · IPC)*	`cl /Zi /O1 pipe_server_min.c /Fe:pipe_server_min.exe`	Inspeccionar flags de `CreateNamedPipeW` (`PIPE_REJECT_REMOTE_CLIENTS`) y la máquina de estados en `\\.\pipe\BlocLabPipe` (4.5 y 2D.10).

Hoja 1 · Nivel Principiante (Etapas 0, 1, 2A, 3 y 4 base)

1.1 Registros x64 / x86 y regla de solapamiento (Lección 2.1)

64 bits	32 bits	16 bits	8 bits (alto / bajo)	Uso convencional en Windows x64 / x86	Preservación (ABI x64)
`RAX`	`EAX`	`AX`	`AH` / `AL`	Valor de retorno entero o puntero; acumulador aritmético	Volátil (lo pisa la función llamada)
`RCX`	`ECX`	`CX`	`CH` / `CL`	**1.er argumento en x64**; `this` en MSVC; contador de turnos/bucles	Volátil
`RDX`	`EDX`	`DX`	`DH` / `DL`	**2.º argumento en x64**; parte alta en `MUL` / `DIV`	Volátil
`RBX`	`EBX`	`BX`	`BH` / `BL`	Registro general preservado (suele guardar punteros o variables de bucle)	**No volátil** (debe restaurarse)
`RSP`	`ESP`	`SP`	`SPL`	Puntero al tope actual del stack (crece hacia direcciones menores)	**No volátil** (alineado a 16B antes de `CALL`)
`RBP`	`EBP`	`BP`	`BPL`	Puntero de marco de función (en x86: argumentos en `[ebp+8...]`, locales en `[ebp-...]`)	**No volátil**
`RSI` / `RDI`	`ESI` / `EDI`	`SI` / `DI`	`SIL` / `DIL`	Origen y destino en operaciones de bloques (`rep movsb`); variables locales	**No volátiles** en Windows
`R8`, `R9`	`R8D`, `R9D`	`R8W`, `R9W`	`R8B`, `R9B`	**3.er y 4.º argumento entero/puntero en x64**	Volátiles
`R10`, `R11`	`R10D`, `R11D`	`R10W`, `R11W`	`R10B`, `R11B`	Temporales de llamada / `syscall`	Volátiles
`R12` – `R15`	`R12D` – `R15D`	`R12W` – `R15W`	`R12B` – `R15B`	Registros de propósito general de larga vida en la función	**No volátiles**

Regla de oro en x64: Escribir en un subregistro de **32 bits** (p. ej. `mov eax, 1`) **pone en cero los 32 bits superiores** de `RAX`. Escribir en **16 u 8 bits** (`ax`, `al`) **conserva intactos** los bits restantes.

1.2 Flags y saltos condicionales tras `CMP op1, op2` (Lección 2.4)

Relación lógica (`op1` vs `op2`)	Salto SIN signo (`unsigned` / direcciones)	Salto CON signo (`int` / complemento a 2)	Condición exacta en los flags (`EFLAGS`)
Igual (`op1 == op2`) / Cero	`JE` / `JZ`	`JE` / `JZ`	`ZF = 1` (tras `test eax, eax`: salta si `eax == 0`)
Distinto (`op1 != op2`) / No cero	`JNE` / `JNZ`	`JNE` / `JNZ`	`ZF = 0`
Mayor estricto (`op1 > op2`)	`JA` / `JNBE` (*Above*)	`JG` / `JNLE` (*Greater*)	Sin signo: `CF = 0 y ZF = 0` · Con signo: `ZF = 0 y SF == OF`
Mayor o igual (`op1 >= op2`)	`JAE` / `JNB` / `JNC`	`JGE` / `JNL`	Sin signo: `CF = 0` · Con signo: `SF == OF`
Menor estricto (`op1 < op2`)	`JB` / `JNAE` / `JC` (*Below*)	`JL` / `JNGE` (*Less*)	Sin signo: `CF = 1` · Con signo: `SF != OF`
Menor o igual (`op1 <= op2`)	`JBE` / `JNA`	`JLE` / `JNG`	Sin signo: `CF = 1 o ZF = 1` · Con signo: `ZF = 1 o SF != OF`

1.3 Conversión VA ↔ RVA ↔ File Offset y convenciones de llamada (Lecciones 1.2, 1.4 y 2.5)

```
RVA         = VA_en_memoria - ImageBase_actual
File Offset = RVA - VirtualAddress_sección + PointerToRawData_sección
VA_en_IDA   = Base_en_IDA + (VA_en_x64dbg - Base_en_x64dbg)   [Sincronización rápida con ASLR]
```

Convención	Arquitectura	Dónde van los primeros argumentos	Quién limpia el stack	Detalle clave en el desensamblado
`cdecl`	x86 (32 bits)	Todos en el stack (de derecha a izquierda; el 1.º en `[esp+4]` tras `call`)	El **llamador** (`add esp, N` después del `call`)	Permite funciones variádicas (`printf` , `scanf`).
`stdcall`	x86 (32 bits)	Todos en el stack (de derecha a izquierda)	La **función llamada** (`ret N` , p. ej. `ret 10h` = 4 args)	Estándar de la API Win32 de 32 bits (`MessageBoxA` , `WindowProc`).
`fastcall`	x86 (32 bits)	1.º en `ECX` , 2.º en `EDX` ; del 3.º en adelante en el stack	La función llamada (`ret N` para los del stack)	En **Delphi x86 (`register`)**: 1.º `EAX` , 2.º `EDX` , 3.º `ECX` (2.13).
`thiscall`	x86 (MSVC)	Puntero `this` en `ECX` ; argumentos explícitos en el stack	La función llamada (`ret N`)	En GCC/MinGW x86, `this` se apila como 1.er argumento oculto en el stack.
Microsoft x64	x64 (64 bits)	1.º `RCX` (o `XMM0`), 2.º `RDX` (`XMM1`), 3.º `R8` (`XMM2`), 4.º `R9` (`XMM3`); 5.º+ en `[rsp+20h]`	El llamador (reserva marco fijo en su prólogo y libera en el epílogo)	El llamador reserva siempre **32 bytes (`0x20`) de *Shadow Space*** en `[rsp+8..rsp+20h]` tras el `call` .

Hoja 2 · Nivel Intermedio (Etapas 2B, 2.17, 2.21, 4.2, 4.5, 5.1, 5.3 y 2D.1–2D.5)

2.1 Offsets de `PEB`, `PEB_LDR_DATA` y `LDR_DATA_TABLE_ENTRY` (Lecciones 2.14, 2.15 y 2D.2)

Estructura / Campo	x86 (32 bits)	x64 (64 bits)	Desde `InMemoryOrderLinks` (x86 / x64)	Para qué lo usa el binario o el analista
Puntero al `TEB` / al `PEB`	`fs:[0x18]` / `fs:[0x30]`	`gs:[0x30]` / `gs:[0x60]`	—	Punto de partida de resolución de APIs y chequeos anti-debug
`PEB.BeingDebugged` (1 byte)	`+0x02`	`+0x02`	—	Leído por `IsDebuggerPresent()` (`1` si hay debugger user-mode)
`PEB.Ldr` (`PEB_LDR_DATA*`)	`+0x0C`	`+0x18`	—	Encabezado de las 3 listas circulares de DLLs cargadas
`PEB.ProcessParameters`	`+0x10`	`+0x20`	—	Contiene `ImagePathName` y `CommandLine`
`PEB.NtGlobalFlag` (`DWORD`)	`+0x68`	`+0xBC`	—	Vale `0x70` si el proceso fue creado por un debugger sin Page Heap
`Ldr.InLoadOrderModuleList`	`+0x0C`	`+0x10`	—	Lista en orden de carga (`Flink` apunta a `+0x00` del entry)
`Ldr.InMemoryOrderModuleList`	`+0x14`	`+0x20`	—	Lista más usada en shellcode/PEB walk (`Flink` apunta a `+0x08` / `+0x10`)
`LDR_DATA_TABLE_ENTRY.DllBase`	**`+0x18`**	**`+0x30`**	**`+0x10` (x86) / `+0x20` (x64)**	Base de la DLL en memoria (cabecera `MZ` para ir a la tabla de Exports)
`LDR_DATA_TABLE_ENTRY.FullDllName`	`+0x24`	`+0x48`	`+0x1C` (x86) / `+0x38` (x64)	`UNICODE_STRING` con la ruta completa de la DLL
`LDR_DATA_TABLE_ENTRY.BaseDllName`	`+0x2C`	`+0x58`	`+0x24` (x86) / `+0x48` (x64)	`UNICODE_STRING` con el nombre corto (p. ej. `kernel32.dll`; buffer en `+4` / `+8`)

2.2 Constantes mágicas de criptografía, hashing y encoding (Lección 4.2)

Algoritmo	Constantes / Bytes característicos en memoria o en instrucciones	Patrón estructural en el código desensamblado
AES (Rijndael)	S-box: `63 7C 77 7B F2 6B 6F C5...` · Rcon: `01 02 04 08 10 20 40 80 1B 36`	Tablas de 256 bytes (o T-tables de 1 KB) o instrucciones nativas `aesenc` / `aesdec` / `aeskeygenassist`.
ChaCha20 / Salsa20	ASCII `"expand 32-byte k"` (`0x61707865, 0x3320646e, 0x79622d32, 0x6b206574`)	Sin tablas; estado de 16 `DWORD`s (64B) y 20 rondas ARX (`add`, `xor`, `rol` de **16, 12, 8 y 7 bits**).
RC4	Sin constantes fijas; arreglo `S[256]` inicializado con `0x00..0xFF`	Dos bucles de 256 iteraciones (KSA con módulo `& 0xFF` e intercambio de bytes + PRGA con `xor` al flujo).
TEA / XTEA	Delta áureo: `0x9E3779B9` (o resta de `0x61C88647`); suma inicial `0xC6EF3720` (32 rondas)	Bucle de 32 o 64 iteraciones que combina `(v << 4) ^ (v >> 5)` y suma el delta en cada vuelta.
MD5 / SHA-1	IV: `0x67452301`, `0xEFCDAB89`, `0x98BADCFE`, `0x10325476` (+ `0xC3D2E1F0` en SHA-1)	Procesamiento en bloques de 64 bytes (64 pasos en MD5; 80 pasos con rotaciones en SHA-1).
SHA-256	IV: `0x6A09E667`, `0xBB67AE85`, `0x3C6EF372`, `0xA54FF53A...` · `K[0] = 0x428A2F98`	Tabla `K[64]` de 64 `DWORD`s y 64 rondas sobre 8 registros de estado (`a..h`).
CRC32	Polinomio reflejado: `0xEDB88320` (segunda entrada de tabla: `0x77073096`), IV `0xFFFFFFFF`	Bucle por byte con `crc = (crc >> 8) ^ tabla[(crc ^ byte) & 0xFF]` y `not crc` al final.
Base64	Tabla de 64 caracteres `"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"` y `'='`	Toma grupos de 3 bytes (24 bits) y los divide en 4 índices de 6 bits (`>> 18`, `>> 12`, `>> 6`, `& 0x3F`).

2.3 Expresiones de x64dbg, scripting y Frida (Lecciones 2.6 y 2.21)

```
# Breakpoint condicional universal (x86/x64) en WindowProc para botón ID 0x13A (WM_COMMAND = 0x111):
breakif:  arg.get(1) == 0x111 && (arg.get(2) & 0xFFFF) == 0x13A
log:      "WM_COMMAND control={arg.get(2)&0xFFFF} notif={arg.get(2)>>16} hwnd={arg.get(0)}"

# Formateadores de log sin detener la ejecución (Break Condition = 0) en x64dbg:
ASCII: {s:rcx}   |   UTF-16: {utf16@rcx}   |   Bytes hex: {mem;16@rcx}   |   Retorno: {rax}

# Plantilla mínima de Frida (guardar como hook.js y correr: frida -f app.exe -l hook.js):
const addr = DebugSymbol.fromName("CreateFileW").address; // o base.add(0x1420)
Interceptor.attach(addr, {
  onEnter(args) { this.path = args[0].readUtf16String(); },
  onLeave(retval) { console.log("CreateFileW:", this.path, "-> handle:", retval); }
});
```

Hoja 3 · Nivel Avanzado (Etapas 2.18–2.20, 2.24, 5.2, 2D.6–2D.13 y Etapa 6)

3.1 Layout en memoria de C++ en MSVC x64: RTTI, SSO, STL y excepciones (Lección 2.18)

Estructura C++ (MSVC x64 Release)	Tamaño total	Desplazamientos y campos internos	Cómo reconocerla en el listado
Objeto con métodos virtuales + RTTI	`8B + campos`	`+0x00`: `vptr` (apunta a `vtable[0]` en `.rdata`). En `vtable[-1]` (`[vptr − 8]`) hay un puntero a `_RTTICompleteObjectLocator`: `+0x00 signature (1)`, `+0x04 offset`, `+0x0C pTypeDescriptor (RVA)`, `+0x10 pClassDescriptor (RVA)`, `+0x14 pSelf (RVA)`.	Llamada `mov rax, [rcx] ; call qword ptr [rax + 8*i]`. Sumar `ImageBase` a `pTypeDescriptor` lleva a `".?AVNombreClase@@"`.
`std::string` (SSO)	`32 bytes` (`0x20`)	`+0x00..+0x0F`: unión `_Bx` (16B buffer local `_Buf` si `_Myres < 16`, o puntero `_Ptr` de 8B en `+0x00` si `_Myres >= 16`) · `+0x10`: `_Mysize` (8B) · `+0x18`: `_Myres` (8B, capacidad).	Comparación `cmp qword ptr [rcx+18h], 10h` seguida de un `cmovae rcx, [rcx]` antes de leer los caracteres.
`std::wstring` (SSO UTF-16)	`32 bytes` (`0x20`)	Igual layout de 32B, pero cada carácter ocupa 2 bytes: el umbral de SSO es `_Myres < 8` caracteres (`cmp [rcx+18h], 8`).	`cmp qword ptr [rcx+18h], 8` antes de elegir entre buffer inline y puntero `_Ptr`.
`std::vector<T>`	`24 bytes` (`0x18`)	`+0x00`: `_Myfirst` (inicio) · `+0x08`: `_Mylast` (fin de usados) · `+0x10`: `_Myend` (fin de reserva).	Cantidad de elementos = `(_Mylast − _Myfirst) / sizeof(T)`; bucle que itera desde `[rcx]` hasta `[rcx+8]`.
Excepciones x64 (`.pdata` / `.xdata`)	12B por entrada `.pdata`	`RUNTIME_FUNCTION`: 3 RVAs de 4B (`BeginAddress`, `EndAddress`, `UnwindData`). En `.xdata`, el handler apunta a `__CxxFrameHandler3` (`FuncInfo` con firma `0x19930520`) o `__CxxFrameHandler4` (`FuncInfo4` comprimido en VS 2019 16.3+/2022).	No hay cadena `FS:[0]` en el prólogo x64; las tablas viven fuera del flujo de código en `.pdata` y `.xdata`.

3.2 Desglose de `CTL_CODE` (32 bits) y métodos de IOCTL en drivers KMDF/WDM (Lecciones 6.2 y 6.4)

```
 31                 16 15    14 13                  2 1      0
┌─────────────────────┬────────┬──────────────────────┬────────┐
│   DeviceType (16b)  │ Access │    Function (12b)    │ Method │
│ 0x8000+ = fabricante│  (2b)  │ 0x800+ = fabricante  │  (2b)  │
└─────────────────────┴────────┴──────────────────────┴────────┘
Fórmula: (DeviceType << 16) | (Access << 14) | (Function << 2) | Method
  Access: 0 = FILE_ANY_ACCESS | 1 = FILE_READ_DATA | 2 = FILE_WRITE_DATA | 3 = READ | WRITE
  Method: 0 = METHOD_BUFFERED (SystemBuffer compartido; seguro salvo punteros embebidos)
          1 = METHOD_IN_DIRECT  (entrada en SystemBuffer; 2.° buffer en Irp->MdlAddress p/lectura)
          2 = METHOD_OUT_DIRECT (entrada en SystemBuffer; 2.° buffer en Irp->MdlAddress
p/escritura)
          3 = METHOD_NEITHER    (Type3InputBuffer y UserBuffer son punteros crudos de user-mode;
                                 exige EvtIoInCallerContext + ProbeAndLock + captura única)
```

3.3 Niveles de `IRQL` y comandos esenciales de WinDbg (Lecciones 2.19, 2D.2, 2D.11, 6.1 y 6.3)

IRQL	Contexto típico de ejecución	Memoria permitida	Sincronización y esperas permitidas
`PASSIVE_LEVEL (0)`	Hilos de usuario, `DriverEntry`, `EvtDriverDeviceAdd`, mayoría de callbacks de archivos/registro	`PagedPool` y `NonPagedPoolNx`	Puede esperar (`KeWaitForSingleObject` con timeout != 0), usar mutexes/eventos y tocar código/datos paginables.
`APC_LEVEL (1)`	Regiones críticas / Guarded mutexes (`Fast Mutex`)	`PagedPool` y `NonPagedPoolNx`	APCs normales de hilo deshabilitadas; page faults aún permitidos.
`DISPATCH_LEVEL (2)`	DPCs (`EvtInterruptDpc`), código bajo `KSPIN_LOCK` (`KeAcquireSpinLock`)	**Solo** `NonPagedPoolNx` (tocar memoria paginada = bugcheck `0x0A` / `0xD1`)	**Prohibido esperar** con intervalo no nulo (solo timeout = 0 y spinlocks). El despachador de hilos no puede cambiar de hilo en ese procesador.
`DIRQL (> 2)`	Rutinas de servicio de interrupción de hardware (`ISR` / `EvtInterruptIsr`)	**Solo** `NonPagedPoolNx`	Solo spinlocks de interrupción en su propio `DIRQL`; delegar el trabajo pesado a un DPC (`DISPATCH_LEVEL`).

```
# 1. Triage de excepción o volcado (User-mode / Kernel-mode – Lecciones 2.19, 2D.11 y 6.3):
!analyze -v                  ; análisis automático del fallo / bugcheck
.exr -1                      ; muestra el EXCEPTION_RECORD actual (código y parámetros)
.ecxr                        ; restaura el contexto de registros en el instante exacto de la
excepción
k   /   kv                   ; stack trace actual   |   ln <addr> : símbolo más cercano a una
dirección

# 2. Diagnóstico de Heap, Memoria, PEB y WOW64 (Lecciones 2.19 y 2D.2):
!heap                        ; lista los heaps del proceso e indica Type (NT Heap vs Segment Heap)
!heap -p -a <addr>           ; con Full Page Heap activo: muestra el call stack que reservó/liberó
el bloque
!vadump                      ; lista todas las regiones VAD en user-mode (Image / Mapped /
Private)
!peb   /   !teb   /   lm     ; inspecciona PEB, TEB y lista de módulos cargados
!wow64exts.sw  /  !wow64exts.k ; alterna contexto 32/64 bits y muestra pila combinada en procesos
WOW64

# 3. Time-Travel Debugging (TTD) y Kernel/WDF (Lecciones 2.19, 6.2 y 6.3):
g-   /   p-   /   t-         ; ejecuta hacia atrás / paso atrás sobre una traza TTD (.run)
ba w4 <addr>  ;  g-          ; frena en la instrucción pasada exacta que sobrescribió esos 4 bytes
dx @$cursession.TTD.Calls("ntdll!NtCreateFile")  ; consulta todas las llamadas a una API en la
traza
!drvobj <driver> 7           ; muestra el DRIVER_OBJECT y su tabla de rutinas MajorFunction
(IRP_MJ_*)
!wdfkd.wdflogdump <driver>   ; vuelca el In-flight Recorder (IFR) de un driver KMDF
```

Hoja 4 · Evasión User-Mode, Inyección, Go/Rust y Extracción In-Place (Rev. 5.4)

4.1 Stubs de Syscall en `ntdll.dll` (x64) y Matriz de Evasión (Lecciones 2.14 y 2.15)

Stub original en `ntdll.dll` (32 bytes)	Stub Indirect Syscall (en el binario)

`4C 8B D1  mov r10, rcx`
`B8 [SSN 4B]  mov eax, <SSN>`
`F6 04 25 ...  test byte ptr [...], 1`
`75 03  jne short +3`
`0F 05  syscall`
`C3  ret`
	`4C 8B D1  mov r10, rcx`
	`B8 [SSN 4B]  mov eax, <SSN>`
	`FF 25 [offset]  jmp qword ptr [pfn_ntdll_syscall]`
	`(salta al 0F 05 dentro de ntdll)`

- **Hell's Gate:** Busca `4C 8B D1 B8` en el export de `ntdll`. **Halo's Gate:** Si el byte `+0` es `0xE9` (hook), busca en vecinos a `±32 * i` bytes. **Tartarus' Gate:** Detecta hooks en el byte `+3` (tras `4C 8B D1`).
- **Detección Defensiva:** 1) `RIP` en `KiSystemCall64` fuera de `ntdll.dll` / `win32u.dll` (caza Direct Syscalls vía `ETWti` / `InstrumentationCallback`). 2) Desenrollado de pila vía `.pdata` donde el llamador de `ntdll` no es `kernelbase.dll` (caza Indirect Syscalls sin Call Stack Spoofing).

4.2 Firma Rápida de Inyección de Procesos en Memoria (Lección 2.22)

Técnica	Secuencia de APIs característica	Punto exacto de intercepción / Evidencia
CreateRemoteThread	`OpenProcess` → `VirtualAllocEx` → `WriteProcessMemory` → `CreateRemoteThread`	Breakpoint en `WriteProcessMemory` (`R8` = buffer local en x64) y `CreateRemoteThread`.
Process Hollowing (RunPE)	`CreateProcessW(CREATE_SUSPENDED=4)` → `NtUnmapViewOfSection` → `VirtualAllocEx` → `WriteProcessMemory` → `SetThreadContext` → `ResumeThread`	En `SetThreadContext`, el nuevo `EntryPoint` se escribe en `ctx.Rcx` (x64, offset `+0x80` de `CONTEXT`) o `ctx.Eax` (x86, offset `+0xB0`).
Early Bird APC	`CreateProcessW(CREATE_SUSPENDED)` → `VirtualAllocEx` → `WriteProcessMemory` → `QueueUserAPC` → `ResumeThread`	Primer argumento de `QueueUserAPC` (`RCX` en x64) apunta al payload inyectado.
Reflective DLL	Reserva memoria `RWX` / `RX` y salta a `ReflectiveLoader` sin `LoadLibrary`	La DLL no figura en `PEB->Ldr` (`lm`), pero existe como región `Private` ejecutable en `!vadump`.

4.3 Go (1.17+ ABI) y Rust (Mangling v0 y Niche Optimization) (Lección 2.13)

- **Go `pclntab` Magic Bytes:** `0xFFFFFFFB` (Go 1.2), `0xFFFFFFFA` (Go 1.16), `0xFFFFFFF0` (Go 1.18+), `0xFFFFFFF1` (Go 1.20+). Recuperar nombres con `GoReSym` o Ghidra 11+.
- **Go `string` y Registros:** Par `{ptr, len}` de 16B **sin byte** `0x00`. Registros de argumentos/retorno en orden: **`RAX, RBX, RCX, RDI, RSI, R8, R9, R10, R11`**. Goroutine activa en **`R14`** (`cmp rsp, [r14+10h]`).
- **Rust `Option<T>`:** Con tipos completos (`Option<u32>`) lleva un `tag` en `+0x00` (`0=None, 1=Some`). Con punteros no nulos (`Option<&T>`, `Option<Box<T>>`, `NonZeroU32`), aplica **Niche Optimization**: suprime el `tag` y compila como `test rax, rax ; jz es_none`.

4.4 Doble Breakpoint Encadenado en `x64dbg` para Decodificadores `void` In-Place (Lección 2.6)

```
1) En el prólogo de la función decodificadora:  Break condition: 0  |  Command: $ptr = rcx
2) En la instrucción RET de la función:         Break condition: 0  |  Log:     "Str: {utf8@$ptr}"
```

Hoja 5 · Criptografía Nativa (CryptoAPI / CNG) y Reversing de Drivers .sys (Rev. 5.4)

5.1 CryptoAPI (`advapi32.dll`) vs. CNG (`bcrypt.dll`) y Endianness (Lección 4.2)

Propiedad	CryptoAPI Clásica (`advapi32.dll`)	CNG Moderna (`bcrypt.dll`)
Buffer de texto plano al cifrar	`CryptEncrypt` : 5.º arg `pbData` (**`[rsp+28h]`** en x64, *in-place*)	`BCryptEncrypt` : 2.º arg `pbInput` (**`RDX`** en x64) y largo en **`R8D`**
Encabezado de clave y Magic	`BLOBHEADER` (8B) + `RSAPUBKEY` (`"RSA1"` = `0x31415352` , `"RSA2"` = `0x32415352`)	`BCRYPT_RSAKEY_BLOB` (`"RSA1"` / `"RSA2"`) · ECC P-256: `"ECK1"` (`0x314B4345`) / `"ECS1"` (`0x31534345`)
Orden de bytes del Módulo RSA	**Little-Endian** (invertir con `[::-1]` antes de exportar a PEM)	**Big-Endian** (orden de red estándar RFC)
Firma ASN.1 DER (X.509 RSA)	`30 82 .. .. 30 0D 06 09 2A 86 48 86 F7 0D 01 01 01 05 00 03 82 .. .. 00 30 82`	

5.2 Offsets Críticos para Triage Estático de Drivers y RPC en x64 (Etapa 6 y Rama 2D)

Estructura (x64)	Offset	Campo y significado en IDA / Ghidra / WinDbg

`DRIVER_OBJECT` (`RCX` en `DriverEntry`)	`+0x68`
	`+0x70`
	`+0xE0`
		`DriverUnload`
		`MajorFunction[0]` (`IRP_MJ_CREATE`)
		`MajorFunction[14]` (**`IRP_MJ_DEVICE_CONTROL`** = `0x70 + 14*8`)

`WDF_DRIVER_CONFIG`	`+0x08`	`EvtDriverDeviceAdd` (pasado a `WdfDriverCreate`)
`WDF_IO_QUEUE_CONFIG`	`+0x28`	**`EvtIoDeviceControl`** (pasado a `WdfIoQueueCreate`)
`FLT_REGISTRATION`	`+0x20`	`OperationRegistration` → arreglo `FLT_OPERATION_REGISTRATION` (`+0x04` Flags, `+0x08` PreOp, `+0x10` PostOp)
`RPC_SERVER_INTERFACE` (`RCX` en `RpcServerRegisterIf*`)	`+0x04` `+0x50`	`InterfaceId` (`RPC_SYNTAX_IDENTIFIER` : UUID de 16B + versión) `InterpreterInfo` → apunta a `MIDL_SERVER_INFO` , cuyo **`+0x08`** (`DispatchTable`) es el arreglo de punteros a las funciones C del servidor RPC.